

MANIPAL UNIVERSITY JAIPUR

Department of Computer Science and Engineering

A DBMS MINI PROJECT REPORT ON

ShopEasy: E-Commerce Website using Python Flask and MySQL

Submitted in partial fulfilment of the requirements of
Bachelor of Technology (Computer Science and Engineering) - 3rd Semester

Submitted by: [Your Full Name]

Registration No.: [Your Reg. No.]

Section / Roll No.: [Section / Roll No.]

Under the guidance of: [Teacher's Name]

Academic Year 2026 - 2027

CERTIFICATE

This is to certify that the project report entitled "**ShopEasy: E-Commerce Website using Python Flask and MySQL**" is a bonafide work carried out by (Registration No.), a student of B.Tech Computer Science and Engineering, 3rd Semester, Manipal University Jaipur, in partial fulfilment of the requirements for the Database Management Systems course during the academic year 2026-2027.

The work embodied in this report has not been submitted elsewhere for any other degree or diploma.

Project Guide

Head of Department

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, , for the guidance and encouragement given throughout this project. I also thank the Department of Computer Science and Engineering, Manipal University Jaipur, for providing the laboratory facilities needed to complete this work.

I am also thankful to my friends and family for their constant support.

ABSTRACT

Online shopping is now a part of daily life. This project, ShopEasy, is a small e-commerce website where a user can register, log in, search products, add them to a cart, place an order and pay. All the data is stored in a MySQL database that has six tables: users, seller, product, orders, orderitem and payment. The tables are connected with primary keys and foreign keys, as shown in the ER diagram.

The backend is written in Python with the Flask framework and the pages are made with HTML and CSS. The project uses DBMS topics like DDL and DML commands, joins, GROUP BY, constraints and transactions. When an order is placed, the orders, orderitem, product and payment tables are changed together in one transaction, so either everything is saved or nothing is. Passwords are stored as hashes and queries use parameters to avoid SQL injection.

The result is a simple working prototype that shows how a web page, Python code and a database work together.

Keywords: E-commerce, DBMS, MySQL, Flask, Python, SQL, Transaction, Normalization.

TABLE OF CONTENTS

1. Introduction.....	6
1.1 Background.....	6
1.2 Problem Statement.....	6
1.3 Objectives.....	6
1.4 Scope.....	6
2. System Requirements.....	7
2.1 Software Requirements.....	7
2.2 Hardware Requirements.....	7
3. Technologies Used.....	8
4. System Design.....	9
4.1 System Architecture.....	9
4.2 ER Diagram.....	9
4.3 Relationships.....	10
4.4 Data Dictionary.....	10
4.5 Normalization.....	11
5. Implementation.....	12
5.1 Database Implementation.....	12
5.2 Backend Modules.....	12
5.3 Frontend Pages.....	12
5.4 Placing an Order (Transaction).....	12
5.5 Security Measures.....	13
5.6 Business Rules.....	13
6. Testing.....	14
7. Screenshots.....	15
8. Limitations and Future Scope.....	18
8.1 Limitations.....	18
8.2 Future Scope.....	18
9. Conclusion.....	19
References.....	20

(This contents list is normal text. If you change the report, update the page numbers by hand.)

1. Introduction

1.1 Background

E-commerce means buying and selling goods over the internet. Websites such as Amazon and Flipkart store a huge amount of data about customers, products, orders and payments. A Database Management System (DBMS) is needed to store this data safely and to get it back quickly. This project builds a small version of such a website to understand how a database and a web application work together.

1.2 Problem Statement

Small shops often keep records in notebooks or Excel sheets. It is hard to search products, check stock, find old orders and avoid billing mistakes. A simple system is needed where customers can order online and all records are kept properly in one place.

1.3 Objectives

- To design a database for an online shop using an ER diagram.
- To implement the database in MySQL using DDL and DML commands.
- To build a website where users can register, log in, search products and use a cart.
- To place orders and record payments using a transaction so that data stays consistent.
- To show reports using JOIN, GROUP BY and aggregate functions.
- To use basic security: password hashing and parameterized queries.

1.4 Scope

The project has registration and login, product list with search, a shopping cart, checkout with payment, order history, a seller list and a small sales report. Real payment, delivery tracking and an admin panel are not included.

2. System Requirements

2.1 Software Requirements

Software	Purpose / Version
Operating System	Windows 10/11, Linux or macOS
Python	Version 3.9 or higher
Flask	Web framework (backend)
MySQL Server	Version 8.x (database)
mysql-connector-python	Connects Python with MySQL
MySQL Workbench	To run SQL scripts and view tables
Web browser	Chrome / Edge / Firefox
Code editor	VS Code or any text editor

2.2 Hardware Requirements

Component	Minimum
Processor	Intel i3 or equivalent
RAM	4 GB
Storage	1 GB free space
Display	Any standard monitor

3. Technologies Used

Python and Flask: Python is an easy language to learn. Flask is a light framework in which every web page is linked to a Python function called a route. It is suitable for beginners because it needs very little setup.

MySQL: A popular open-source relational database. Data is kept in tables and is handled with SQL. It supports primary keys, foreign keys, constraints and transactions.

HTML and CSS: HTML builds the structure of the pages and CSS gives them colours and layout. Flask fills the HTML templates with data using the Jinja2 template language.

Werkzeug security: Used to convert passwords into hashes and to check them at login.

4. System Design

4.1 System Architecture

The project follows a simple three-part structure. The user works in the browser (frontend). Each click sends a request to the Flask application (backend). Flask runs the SQL queries on the MySQL database, gets the result and returns an HTML page to the browser.

Layer	Technology	Work done
Frontend	HTML, CSS, Jinja2 templates	Shows pages and forms to the user
Backend	Python, Flask	Handles requests, checks input, runs business rules
Database	MySQL	Stores and retrieves all data

4.2 ER Diagram

The ER diagram below shows the six tables and how they are connected. A seller lists many products, a user places many orders, an order contains many order items and is paid by one payment, and a product can appear in many order items.

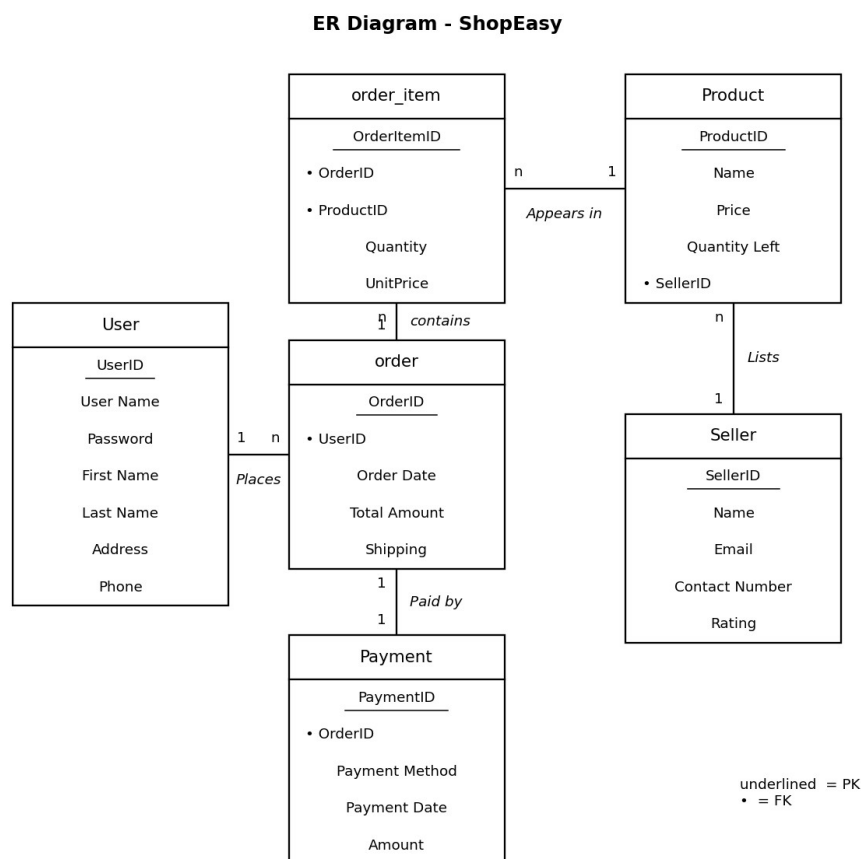


Figure 4.1: ER diagram of ShopEasy (names as in the ER diagram; the SQL tables are users, seller, product, orders, orderitem, payment)

4.3 Relationships

Relationship	Type	Foreign key used
seller lists product	1 : n	product.sellerid
user places order	1 : n	orders.userid
order contains order items	1 : n	orderitem.orderid
product appears in order items	1 : n	orderitem.productid
order is paid by payment	1 : 1	payment.orderid

4.4 Data Dictionary

The structure of every table is given below.

Table 4.1: users

Column	Data type	Constraint	Description
userid	INT	PRIMARY KEY, AUTO_INCREMENT	Unique id of the customer
username	VARCHAR(45)	NOT NULL, UNIQUE	Login name
firstname	VARCHAR(45)	NOT NULL	First name
lastname	VARCHAR(45)	NOT NULL	Last name
address	VARCHAR(120)	NOT NULL	Delivery address
mobile	BIGINT	NOT NULL	10-digit mobile number
email	VARCHAR(100)	-	Email address
password	VARCHAR(255)	NOT NULL	Hashed password

Table 4.2: seller

Column	Data type	Constraint	Description
sellerid	INT	PRIMARY KEY	Unique id of the seller
sname	VARCHAR(45)	NOT NULL	Shop name
email	VARCHAR(45)	NOT NULL	Seller email
rating	DECIMAL(2,1)	-	Rating out of 5
phone	BIGINT	NOT NULL	Contact number

Table 4.3: product

Column	Data type	Constraint	Description
productid	INT	PRIMARY KEY	Unique id of the product
pname	VARCHAR(45)	NOT NULL	Product name
price	FLOAT	NOT NULL	Price in rupees
qty	INT	NOT NULL	Stock available
sellerid	INT	FOREIGN KEY -> seller	Who sells the product

Table 4.4: orders

Column	Data type	Constraint	Description
orderid	INT	PRIMARY KEY, AUTO_INCREMENT	Unique order number
userid	INT	FOREIGN KEY -> users	Customer who ordered
orderdate	DATETIME	-	Date and time of order
total	DECIMAL(10,2)	NOT NULL	Total price of the items
shipping	VARCHAR(6)	NOT NULL	FREE or 50

Table 4.5: orderitem

Column	Data type	Constraint	Description
orderitemid	INT	PRIMARY KEY, AUTO_INCREMENT	Unique id of the line
orderid	INT	FOREIGN KEY -> orders	Order it belongs to
productid	INT	FOREIGN KEY -> product	Product bought
quantity	INT	NOT NULL	Pieces bought
unitprice	DECIMAL(10,2)	NOT NULL	Price at time of buying

Table 4.6: payment

Column	Data type	Constraint	Description
paymentid	INT	PRIMARY KEY, AUTO_INCREMENT	Unique payment id
paymethod	VARCHAR(20)	NOT NULL	UPI / Card / Net Banking / COD
paydate	DATETIME	-	Date and time of payment
amount	DECIMAL(10,2)	NOT NULL	Amount paid
orderid	INT	FOREIGN KEY -> orders	Order paid for

4.5 Normalization

- 1NF: every column has one value. The products of an order are not kept in one cell; each product is a separate row in orderitem.
- 2NF: columns depend on the full primary key. Product name and price are only in product, not repeated in orderitem.
- 3NF: no column depends on another non-key column. Seller details are in seller and product only keeps sellerid.

unitprice is also saved in orderitem on purpose. The price of a product can change later, but an old order must show the price at which it was bought.

5. Implementation

5.1 Database Implementation

The database is created by schema.sql. It has CREATE TABLE commands with primary keys, foreign keys, UNIQUE and NOT NULL, and then INSERT commands for sample data. The lab script (lab_script_original.sql) shows ALTER TABLE (add column, rename column, rename table), UPDATE, TRUNCATE and a JOIN of users and orders. queries.sql has more queries: WHERE, ORDER BY, JOIN of three tables, GROUP BY with SUM, COUNT and AVG, subquery, LEFT JOIN and a VIEW.

5.2 Backend Modules

Module	Route	Function
Home	/	Lists products; search by name, filter by seller
Register	/register	Validates input, hashes password, inserts a user
Login / Logout	/login, /logout	Checks the password hash and stores the user in the session
Add to cart	/add_to_cart/<id>	Adds a product to the cart (kept in the session) after checking stock
Cart	/cart, /cart/update, /cart/remove	Shows the cart, changes quantity, removes items
Checkout	/checkout	Saves the order, order items and payment; reduces stock
Orders	/orders, /order/<id>	Order history and details of one order
Sellers	/sellers	Shows sellers with product count
Report	/report	Totals, sales per seller, low stock products

5.3 Frontend Pages

All pages use one common file, base.html, which has the navigation bar, the message area and the footer. The other pages are home, register, login, cart, checkout, orders, order detail, sellers and report. The colours and layout are in style.css.

5.4 Placing an Order (Transaction)

Placing an order changes four tables: orders, orderitem, product (stock) and payment. If one step fails, the earlier steps must be undone, so we use commit and rollback. The cart is not a table; it is kept in the session and is emptied after the order is saved.

```

conn = get_connection()
cursor = conn.cursor()
try:
    cursor.execute("INSERT INTO orders (...) VALUES (...)", ...)
    orderid = cursor.lastrowid
    for item in items:
        cursor.execute("UPDATE product SET qty = qty - %s
                        WHERE productid = %s AND qty >= %s", ...)
        if cursor.rowcount == 0:
            raise ValueError("Not enough stock")
        cursor.execute("INSERT INTO orderitem (...) VALUES (...)", ...)
    cursor.execute("INSERT INTO payment (...) VALUES (...)", ...)
    conn.commit()      # save everything
except Exception:
    conn.rollback()    # undo everything

```

5.5 Security Measures

- Passwords are saved as hashes (generate_password_hash) and checked with check_password_hash.
- All SQL queries use %s placeholders (parameterized queries) to prevent SQL injection.
- Cart, checkout and order pages need login. A user can open only his own orders.
- Inputs are validated: all fields required, mobile number of 10 digits, minimum password length, stock limits.

5.6 Business Rules

- Shipping is free if the value of items is Rs. 5000 or more; otherwise Rs. 50 is charged.
- Order total = price of the items. Shipping (FREE or 50) is shown separately, like in the sample data.
- Stock is reduced when an order is placed. More than the available stock cannot be added to the cart.

6. Testing

The application was tested module by module. The important test cases and their results are given below.

No.	Test case	Expected result	Result
1	Open home page	All 9 products shown	Pass
2	Search 'app'	Only Apple products shown	Pass
3	Filter by seller	Only that seller's products shown	Pass
4	Register with 5-digit mobile	Error message, user not created	Pass
5	Register with existing username	Error: username already taken	Pass
6	Register with valid details	User created, redirect to login	Pass
7	Login with wrong password	Error message shown	Pass
8	Login with correct password	Welcome message, user in session	Pass
9	Open cart without login	Redirected to login page	Pass
10	Add same product twice	Cart shows quantity 2	Pass
11	Update quantity above stock	Error: only N in stock	Pass
12	Checkout with cart value above Rs. 5000	Shipping FREE, order saved	Pass
13	Checkout with small cart	Shipping saved as 50	Pass
14	After order	Stock reduced, cart empty, payment saved	Pass
15	Open another user's order	Order not found message	Pass
16	Stock becomes insufficient during checkout	Order rolled back, nothing saved	Pass

7. Screenshots

Run the project, take screenshots of the following pages and paste them in the boxes below.

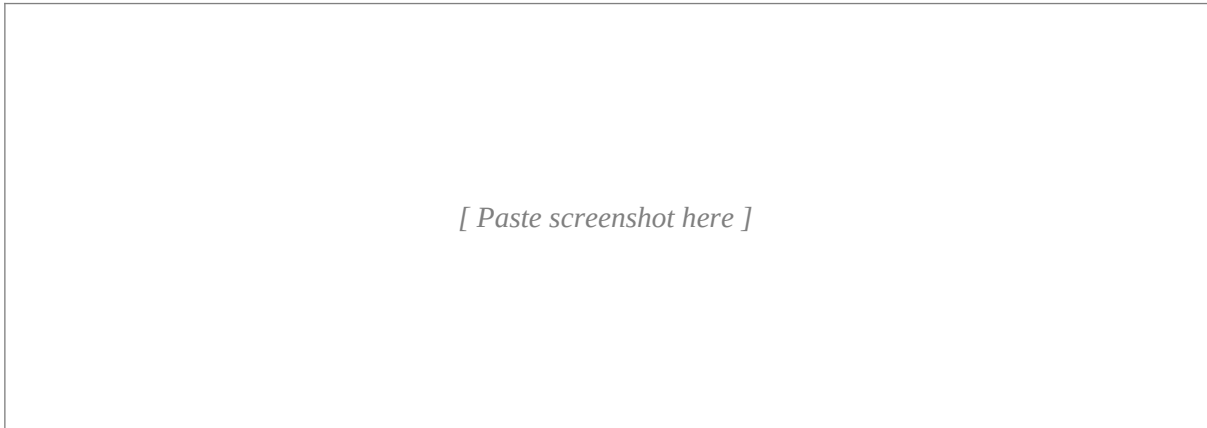


Figure 7.1: Home page with products

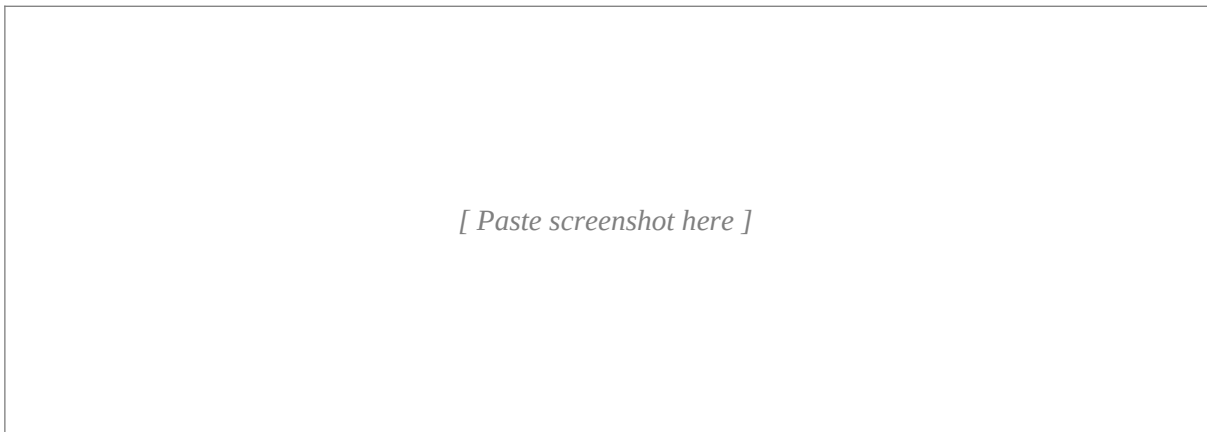


Figure 7.2: Registration page

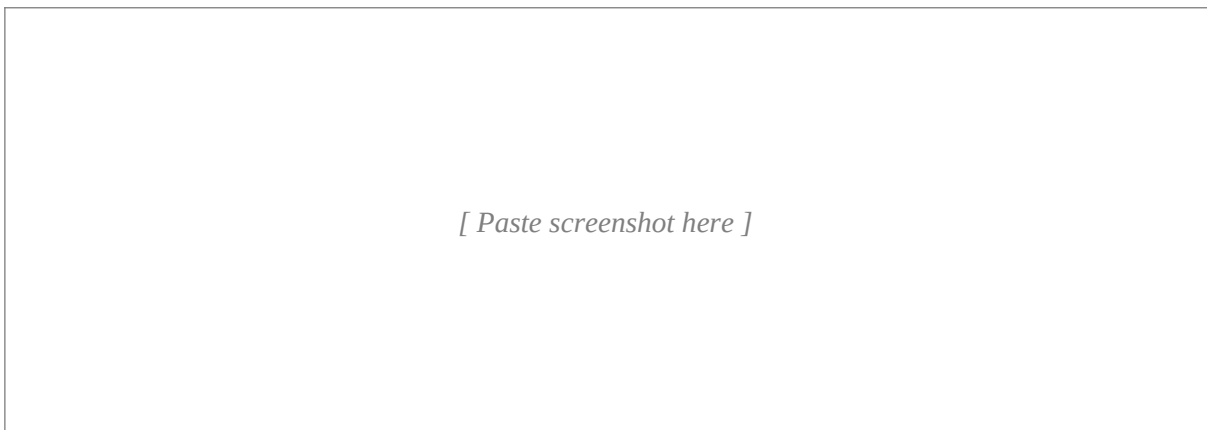


Figure 7.3: Login page

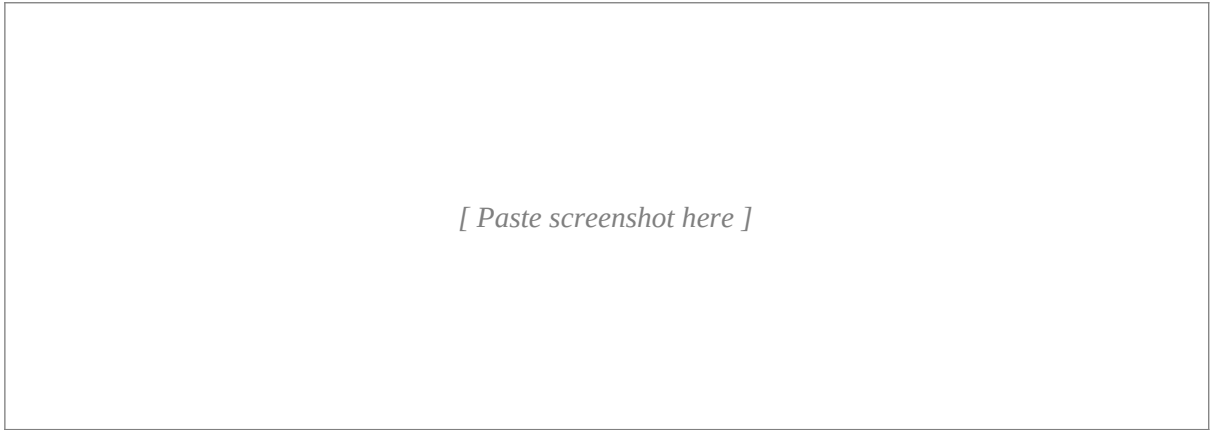


Figure 7.4: Shopping cart

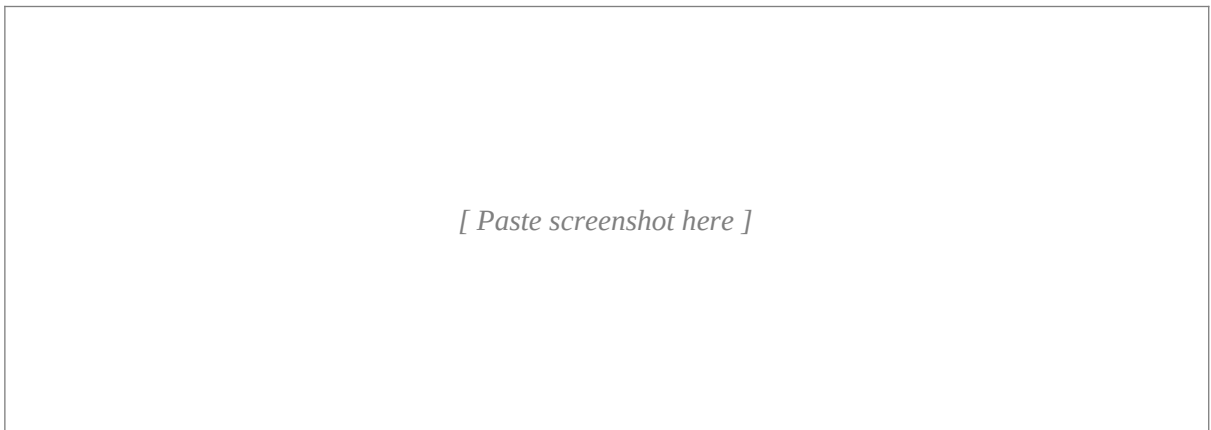


Figure 7.5: Checkout page

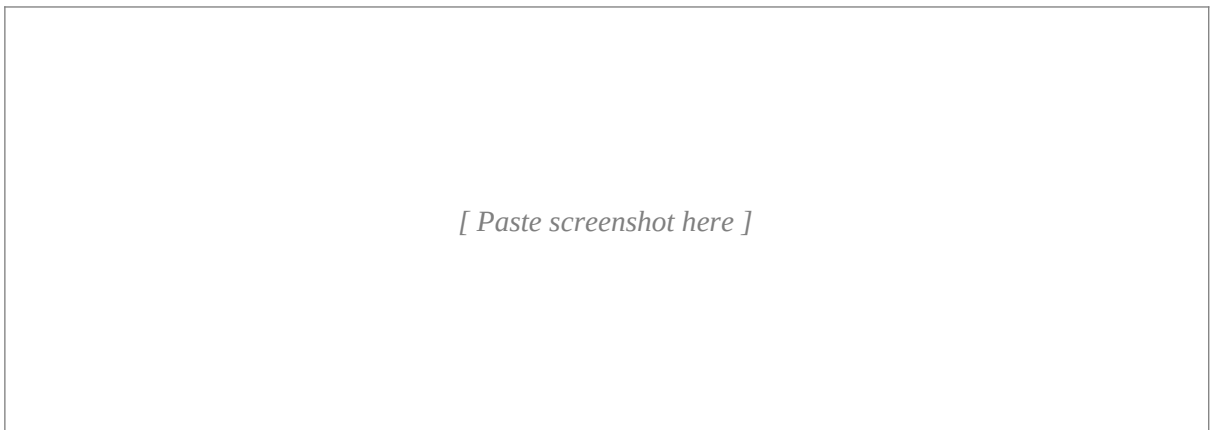


Figure 7.6: Order details

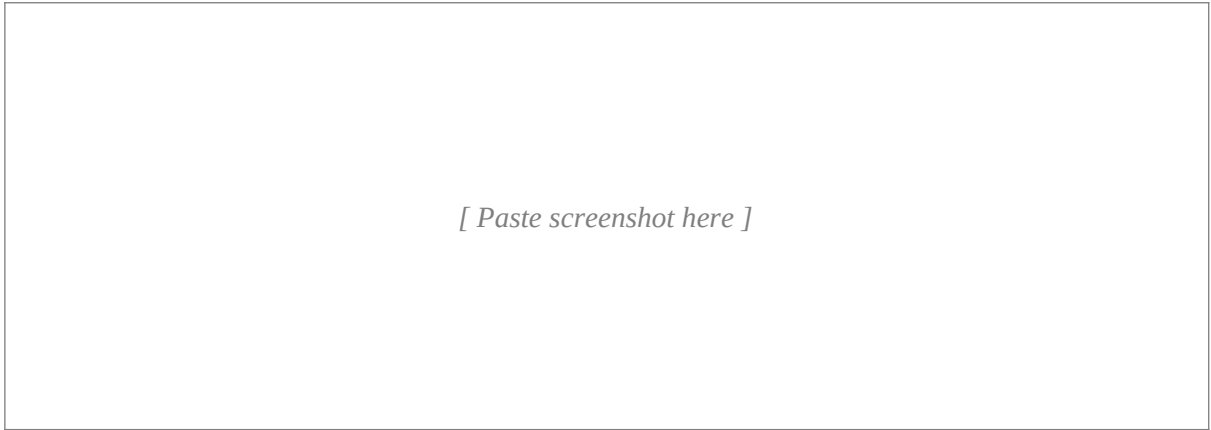


Figure 7.7: Sales report page

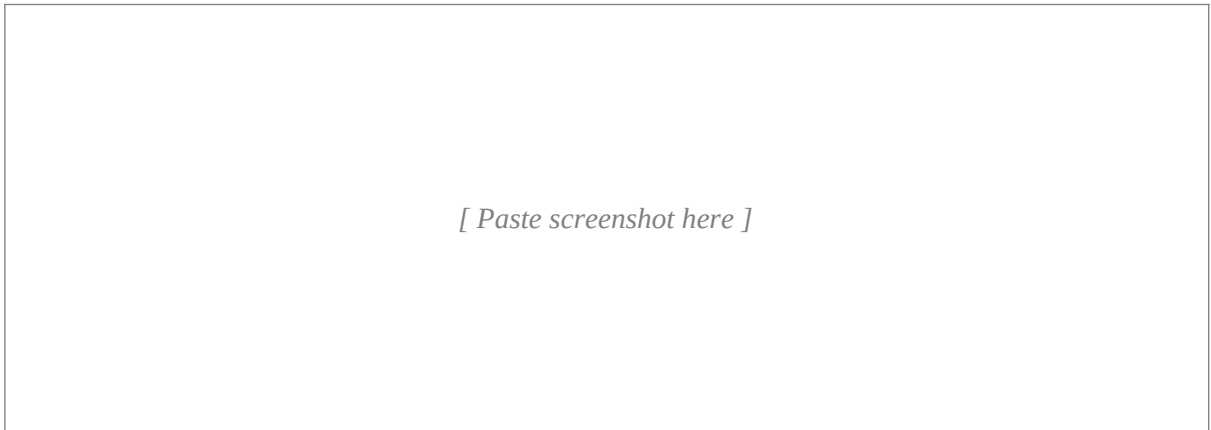


Figure 7.8: Tables in MySQL Workbench

8. Limitations and Future Scope

8.1 Limitations

- Payment is only recorded; no real payment gateway is connected.
- There is no admin login, so the report page is open to everyone.
- Sellers cannot add or edit their products from the website.
- Products have no images, reviews or ratings given by customers.
- A new database connection is opened for every query, which is slow if many users use it.

8.2 Future Scope

- Integrate a payment gateway such as Razorpay.
- Add an admin panel and a seller dashboard.
- Add product images, reviews and a wishlist.
- Add order cancellation, tracking and email notifications.
- Use connection pooling and deploy the project on a cloud server.

9. Conclusion

The ShopEasy project helped me understand how a website and a database work together. A database with six tables was designed from the ER diagram and made in MySQL, and a Flask website was built on it. The project used many topics from our DBMS course: keys and constraints, DDL and DML, joins, GROUP BY, normalization and transactions. I also learnt the basics of web development and security. The prototype works for all the test cases and more features can be added later.

REFERENCES

- [1] Flask Documentation - <https://flask.palletsprojects.com>
- [2] MySQL 8.0 Reference Manual - <https://dev.mysql.com/doc/>
- [3] MySQL Connector/Python Developer Guide - <https://dev.mysql.com/doc/connector-python/en/>
- [4] Jinja2 Template Documentation - <https://jinja.palletsprojects.com>
- [5] W3Schools SQL Tutorial - <https://www.w3schools.com/sql/>
- [6] R. Elmasri and S. Navathe, Fundamentals of Database Systems, Pearson.
- [7] A. Silberschatz, H. Korth and S. Sudarshan, Database System Concepts, McGraw-Hill.